

Problem D. Not a subset sum - 解题报告

1. 题目背景与分析

题目大意：

给定一个长度为 2^n 的数组 a （下标 0 到 $2^n - 1$ ）。对于每一个长度为 n 的字符串 $q \in \{0, 1, ?\}^n$ ，定义“广义子集和” $S(q)$ 为满足以下条件的所有下标 j 的 a_j 之和：

- 如果 q 的第 k 位是 '0'，则 j 的第 k 位必须是 0。
- 如果 q 的第 k 位是 '1'，则 j 的第 k 位必须是 1。
- 如果 q 的第 k 位是 '?'，则 j 的第 k 位没有限制（既可以是 0 也可以是 1）。

目标：

计算所有 3^n 个可能的字符串 q 对应的 $S(q)$ 的按位异或和（XOR sum）。

数据范围：

- $1 \leq n \leq 16$
- $0 \leq a_i \leq 9$

2. 核心算法：逐维扩展 (Dimension Expansion)

暴力法的局限

如果直接枚举所有 3^n 个 q ，对于每个 q 遍历 2^n 个数组元素来求和，总时间复杂度为 $O(3^n \cdot 2^n)$ 。
当 $n = 16$ 时， $3^{16} \approx 4.3 \times 10^7$ ，而 $2^{16} = 65536$ 。乘积约为 2.8×10^{12} ，这在 1 秒内绝对无法完成。

优化思路：分治与独立性

注意到每一个二进制位（维度）对结果的贡献是独立的。这类似于快速沃尔什变换 (FWHT) 或 SOS DP (高维前缀和) 的思想。

我们可以将问题看作从 n 个 $\{0, 1\}$ 的维度，逐步扩展到 n 个 $\{0, 1, ?\}$ 的维度。

变换规则：

假设我们要处理当前这一个维度。对于其他维度完全相同的一对数 v_0 （对应当前位为 0）和 v_1 （对应当前位为 1），它们在扩展后的维度中会产生三个值：

- 取 0：结果为 v_0 （对应查询字符 '0'）。
- 取 1：结果为 v_1 （对应查询字符 '1'）。
- 取 ?：结果为 $v_0 + v_1$ （对应查询字符 '?'，即两个都要）。

算法流程

我们需要进行 n 轮迭代，每一轮处理一个二进制位：

- 初始状态：数组大小为 2^n 。
- 第 1 轮：处理第 1 个 bit。我们将数组中的每对 (v_0, v_1) 替换为 $(v_0, v_1, v_0 + v_1)$ 。此时数组大小变为 $3^1 \times 2^{n-1}$ 。

3. **第 k 轮**: 处理第 k 个 bit。此前已经处理过的 $k - 1$ 个维度已经变成了 3 进制，未处理的维度还是 2 进制。我们将对应的数据块结合，大小变为 $3^k \times 2^{n-k}$ 。
4. **结束状态**: 经过 n 轮后，数组大小变为 3^n ，包含了所有 $S(q)$ 的值。

3. 复杂度分析

时间复杂度

在第 k 步 ($0 \leq k < n$)，输入数组的大小是 $3^k \cdot 2^{n-k}$ ，输出数组的大小是 $3^{k+1} \cdot 2^{n-k-1}$ 。

总的运算量正比于所有步骤输出元素数量之和：

$$T(n) \approx \sum_{k=0}^{n-1} 3^{k+1} \cdot 2^{n-1-k}$$

这是一个公比为 1.5 的等比数列求和，其总和主要由最后一项决定。

$$T(n) = O(3^n)$$

当 $n = 16$ 时， $3^{16} \approx 43,000,000$ ，计算量在 10^8 以内，完全可以在 1 秒内跑完。

空间复杂度

如果直接存储最终结果，需要一个大小为 3^{16} 的 `int` 数组。

$43,000,000 \times 4 \text{ bytes} \approx 164 \text{ MB}$ 。

这在大部分竞赛的内存限制（256MB 或 512MB）内是安全的，但为了更稳妥，我们可以进行**空间优化**：

- **滚动数组**：只需要保存 `current_layer` 和 `next_layer`。
- **最终步优化**：在处理最后一步（第 $n - 1$ 步）时，不需要将结果存入数组，而是直接计算出三个值后进行异或累加。这样最大内存占用仅为第 $n - 1$ 步的大小（约 115 MB）。

4. C++ 代码实现

```
/**
 * Problem D. Not a subset sum
 * Algorithm: Dimension-by-dimension expansion ( $O(3^n)$ )
 */

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
    // 1. 优化 I/O 效率
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    if (!(cin >> n)) return 0;

    // 初始输入大小为  $2^n$ 
    int num_elements = 1 << n;
```

```

// 使用 vector 存储当前层的数据
// 初始层: 所有维度都是二进制 (大小  $2^n$ )
vector<int> current_layer(num_elements);
for (int i = 0; i < num_elements; ++i) {
    cin >> current_layer[i];
}

// 2. 逐个维度进行扩展处理
// step 代表当前正在处理第几个维度 (0 到 n-1)
for (int step = 0; step < n; ++step) {
    // prefix_count: 已经被处理过的维度形成的块的大小 (3的幂次)
    // suffix_count: 尚未处理的维度形成的块的数量 (2的幂次)

    int prefix_count = 1;
    for(int k = 0; k < step; ++k) prefix_count *= 3;

    // 计算当前层未处理的块数:  $2^{(n-1-step)}$ 
    int suffix_count = 1 << (n - 1 - step);

    // stride: 在当前层中, 同一组数据中 bit=0 和 bit=1 的间隔距离
    // 这等于上一轮处理完后的块大小 (prefix_count)
    int stride = prefix_count;

    // === 优化关键点 ===
    // 如果是最后一步 (step == n - 1), 我们不需要构建那个巨大的  $3^n$  数组。
    // 直接计算出结果并异或到 total_xor 中, 节省大量内存和分配时间。
    if (step == n - 1) {
        int total_xor = 0;
        // 此时 suffix_count 为 1, 只有一个大的逻辑块
        for (int p = 0; p < prefix_count; ++p) {
            int v0 = current_layer[p];           // 对应位为 0 的值
            int v1 = current_layer[p + stride]; // 对应位为 1 的值

            // 产生三个结果并异或
            total_xor ^= v0;           // q = ...0...
            total_xor ^= v1;           // q = ...1...
            total_xor ^= (v0 + v1);    // q = ...?...
        }
        cout << total_xor << endl;
        return 0;
    }

    // 3. 构建下一层
    // 下一层的大小将是: suffix_count * 3 * prefix_count
    // 我们使用 reserve 预分配内存以避免扩容开销
    vector<int> next_layer;
    next_layer.reserve(suffix_count * 3 * prefix_count);

    for (int s = 0; s < suffix_count; ++s) {
        // 计算当前未处理后缀块的起始位置
        // 每个后缀块包含两部分: bit=0 的部分和 bit=1 的部分
        // 总长度是 2 * stride
    }
}

```

```
int block_start = s * 2 * stride;

// 遍历已处理的前缀块中的每一个元素，将其扩展
for (int p = 0; p < prefix_count; ++p) {
    int v0 = current_layer[block_start + p];
    int v1 = current_layer[block_start + stride + p];

    // 将 (v0, v1) 扩展为 (v0, v1, v0 + v1)
    next_layer.push_back(v0);
    next_layer.push_back(v1);
    next_layer.push_back(v0 + v1);
}

// 更新 current_layer，利用 move 语义避免拷贝
current_layer = move(next_layer);
}

return 0;
}
```